

Exercise 1:

Try the code and observe the output. Use the NashPy documentation and your favorite generative AI assistant to understand the results.

given the payoff matrix of players A and B:

PA = Matrix([[-1, -3], [0, -2]])

PB = Matrix([[-1, 0], [-3, -2]])

and the print method to understand the output:

```
equilibrium = list(game.support_enumeration())  
  
print(equilibrium)
```

output:

```
jln@kitty:~/decision_making/Lab2_game_theory/  
game_theory/EX1.py  
The prisoner's dilemma  
[(array([0., 1.]), array([0., 1.]))]
```

we observe that the function `support_enumeration()`, returns the nash equilibrium of the players matrixes, then it prints a tuple of arrays, this arrays states the probability in which player A or B is gonna use each strategy, as we can see in the photo, both arrays are the same `[0,1]`, given 0% profitability of using first strategy which in this case is collaborate, and 100% of second strategy, in this case betray.

This means that in order the Nash equilibrium is produced both PA and PB will choose to betray always.

Exercise 2:

Two people like different types of shows, but they like to attend together. For the first person (P1), (s)he likes to go to the theater together (+4). For the second person (P2), (s)he likes to go to the ballet together (+4).

For both people, although they do not like the other show, they value spending time together (+2). If they attend different shows, whether it is his/her favorite show or not, they do not enjoy it at all (+0).

Find the optimal strategies for both people using the NashPy library and explain them. Define the players, the possible strategies, and the payoff matrices.

The payoff matrices (1st strategy is theater, 2nd is ballet)

P1 = np.array([[4, 0], [0, 2]])

P2 = np.array([[2, 0], [0, 4]])

output:

Ballet or theater

```
[(array([1., 0.]), array([1., 0.])),  
 (array([0., 1.]), array([0., 1.])),  
 (array([0.66666667, 0.33333333]), array([0.33333333, 0.66666667]))]
```

Seems we got 3 different strategies.

The first and second strategy look alike, since when choosing different activities the reward is 0, they try to choose to go to only one of them, in the first they always choose theater and in the second they always go to ballet.

The third one however both players are a little more selfish wanting to choose what they like with a higher probability, so the result will more likely be that they go to different shows.

Exercise 3:

Try the code and observe the output. Now compute the utility for both players manually. Do you get the same results?

Yes, we get the same results.

Exercise 4:

Now, compute the utility for both players using one of the results of the Nash equilibrium. Why are the results so different? Can you change the strategy for one of the players without reducing the other player's utility? Show it.

```
game_theory/Ex4.py  
The prisoner's dilemma  
Utility for P1 = -2  
Utility for P2 = -2  
• jin@kitty:~/decision_m  
The prisoner's dilemma  
Utility for P1 = -2.0  
Utility for P2 = -0.5
```

since they always choose to betray in the Nash equilibrium, their utility will always be -2.

to increase said utility we could follow a strategy of only cooperating, since when both cooperate they receive a mild punishment→

when both sigmas are [1,0] or always cooperate the output becomes -1 for both which is higher than before.→

```
Utility for P1 = -1
Utility for P2 = -1
```

Exercise 5:

Try the code and observe the output. Compare it with the Nash equilibria obtained in Exercise 1 and comment on the differences or similarities.

```
[ 1. 95.] [ 0. 96.]
[ 1. 96.] [ 0. 97.]
[ 1. 97.] [ 0. 98.]
[ 1. 98.] [ 0. 99.]
[ 1. 99.] [ 0. 100.]
jin@kitty:~/decision_making/Lab2_game_theory$ /bin/python3
game_theory/EX1.py
The prisoner's dilemma
[(array([0., 1.]), array([0., 1.]))]
```

The output of this exercise shows us the evolution of the different iterations, although is not shown we can see that in the second iteration while player A tried to cooperate player B chose to betray and since then both players always betray each other.

comparing the Nash equilibriums, after the 100 iterations they look roughly the same.

Final Exercise:

Two autonomous robots, Robot A and Robot B, are responsible for three tasks in a warehouse:

- Task 1: Sorting packages.
- Task 2: Delivering packages.
- Task 3: Inspecting equipment.

Each robot is optimized for different tasks:

- Robot A sorts packages in 2 minutes, while Robot B takes 8 minutes.
- Robot A delivers packages in 6 minutes, while Robot B takes 13 minutes.
- Robot A needs 17 minutes for inspecting equipment, while Robot B needs just 3 minutes.

When both robots execute the same task, times are doubled because of the interferences.

You are asked to:

1. **Define the payoff matrices considering:**
 - a) **We want to minimize the time employed for completing the tasks.**
 - b) **The amount of time each robot takes for the different tasks.**

```
RA = np.array([[ -4, -2, -2], [ -6, -12, -6], [ -17, -17, -34]])
```

```
RB = np.array([[ -16, -8, -8], [ -13, -26, -13], [ -3, -3, -6]])
```

2. **Compute the Nash equilibrium. What's the problem with the obtained outcome?**

```
RobotA and RobotB working
/home/jin/.local/lib/python3.8/site-packages/nashpy/algorithms/support_enumeration.py:260:
RuntimeWarning:
An even number of (2) equilibria was returned. This
indicates that the game is degenerate. Consider using another algorithm
to investigate.

warnings.warn(warning, RuntimeWarning)
[(array([1., 0., 0.]), array([0., 1., 0.])), (array([1., 0., 0.]), array([0., 0., 1.]))]
```

Seems like we have an even number of Nash equilibrium and that makes the game degenerate. When a game is degenerate it means that it has multiple Nash equilibrium and there are a lot of ways where we can arrive at the equilibrium point, so there's multiple strategies with roughly the same outcome, resulting in ambiguity and complexity on the players actions.

3. **Use the obtained Nash Equilibrium strategies to simulate how the robots execute the tasks based on those strategies. This code will run 100 rounds where the robots select tasks according to their Nash Equilibrium strategies and observe how tasks are allocated over time. Simulate each task by printing a message.**

```
Strategy 1:((array([1., 0., 0.]), array([0., 1., 0.])))
```

```
Strategy 2:((array([1., 0., 0.]), array([0., 0., 1.])))
```

Since the strategies obtained are clearly defined, RobotA will always sort, and RobotB in the first strategy will always deliver packages while in the second strategy RobotB will always inspect equipment.

4. **Define a new strategy that guarantees that all tasks are completed at least once by each robot. Simulate the robots' decision-making over 100 rounds.**

```
Robot B is following this strategy [0.1,0.1,0.8]
```

```
Robot A is following this aprox [0.9,0.07,0.03]
```

Since we are using probabilities we can't ensure that a 5 percent is enough for robot A to complete both tasks at least once, so I made a simulation of the decision making of these robots in order to increase the efficiency with some rules.

Robot B likes to inspect equipment but is very bad at delivering packages, far worse than robot A at delivering, so with some rules letting robot B choose if he wants to do the sorting, robot A can then choose to do delivering or inspecting, since it is better at delivering he will choose that more often.

However this simulation wasn't done with numpy libraries so the strategy probabilities are an approximation.

5. **Compare the utilities for the Nash Equilibrium strategy and the new strategy, and justify the differences.**

Nash equilibrium strategy:

Utility for RobotA = -2.0

Utility for RobotB = -8.0

Personal strategy:

Utility for RobotA = -4.0600000000000005

Utility for RobotB = -9.785

Although the Nash equilibrium strategy seems to have better utility, this is because in the personal strategy we forced both robots to do all tasks at least once, and since working with probability can result in fuzzy endings we can't ensure that the robot does the task only once, so we increase the probability to a threshold in which the robot ends up taking a task which is very bad at, reducing utility.

6. **Calculate the cumulative payoff for both robots after 100 rounds using the Nash Equilibrium strategy and the new strategy. Explain the differences.**

Nash equilibrium strategy:

Strategy 1: (array([1., 0., 0.]), array([0., 1., 0.]))

Cumulative payoff Robot A = -200

Cumulative payoff Robot B = -1300

Strategy 2: (array([1., 0., 0.]), array([0., 0., 1.]))

Cumulative payoff Robot A = -200

Cumulative payoff Robot B = -300

Personal strategy:

```
Robot A did:[sorting packages: 86 times.  
Delivering packages: 11 times.  
Inspecting equipment 3 times  
  
Robot B did:[sorting packages: 14 times.  
Delivering packages: 13 times.  
Inspecting equipment 73 times  
  
Cumulative payoff Robot A = -117  
Cumulative payoff Robot B = -500
```

7. Now, suppose that robots do not cooperate and operate independently. Modify the code to learn a Nash Equilibrium and compare it to the Nash Equilibrium obtained previously.

If they don't cooperate their payoff matrices would look like:

```
RA = np.array([[-2, -2, -2], [-6, -6, -6], [-17, -17, -17]])
```

```
RB = np.array([[-8, -8, -8], [-13, -13, -13], [-3, -3, -3]])
```

While computing the nash equilibrium we end up with:

```
Strategy 1 = (array([1., 0., 0.]), array([1., 0., 0.]))
```

```
Strategy 2 = (array([1., 0., 0.]), array([0., 1., 0.]))
```

```
Strategy 3 = (array([1., 0., 0.]), array([0., 0., 1.]))
```

In this example we got an odd number of clearly defined strategies, the game is then non degenerate and we have defined 3 clear strategies which they can follow to arrive at a Nash equilibrium point.¹

¹ In the uploaded python files, the File Final_4.py corresponds to the algorithm of the personal strategy, while Final_multiple.py is used for multiple computations required for different exercises.